Summary of ELEC 433

Tom Wang

Spring, 2024

1 Introduction to Hamming Codes

ECC solves the problem of errors in transmission. The idea is to add redundancy to the message so that the receiver can detect and correct errors.

Assume that we have a message of length k, then we encode it to a codeword C of length n. If the codeword is decrypted, then we call it $\tilde{C}$.

1.1 Things we care about

1. **Error Detection**: We want to be able to detect if there are errors in the transmission.
2. **Error Correction**: We want to be able to correct the errors in the transmission.
3. **Efficiency**: We want to be able to minimize the number of bits we need to add to the message. So minimize the overhead. ($\frac{k}{n}$ should be as large as possible)
4. **Complexity**: We want to be able to encode and decode the message in a reasonable amount of time.
5. **Robustness**: We want to be able to correct as many errors as possible.
6. **Flexibility**: We want to be able to correct different types of errors.

1.2 Definitions

First of all, we need to define a code:

Definition 1. *A code C of length n over an alphabet Σ is a subset of Σ^n. So $C \subseteq \Sigma^n$. The elements $c \in C$ are called codewords.*

There are two types of errors:

- **Erasure**: We know which bit is erased, but we don't know what it is. So we can **detect** and **correct** erasures.
- **Error**: We don't know which bit is wrong, but we know that there is at least one bit that is wrong. So we can **detect** errors, but we can't for sure **correct** them.

Also, we define all possible codewords to be C_i where they all belong to the same set $\mathcal{C}$. So $C_i \in \mathcal{C}$ for all possible i.

1.3 Hamming Codes

Hamming codes are linear block codes. They are named after Richard Hamming. They are single-error correcting and double-error detecting. They are also systematic, which means that the message is part of the codeword.

Hamming code encodes a message of length 4 to a codeword of length 7. So $k = 4$ and $n = 7$. Also, it encodes only binary messages, so $\Sigma = \{0, 1\}$.

$$(x_1, x_2, x_3, x_4) \implies (x_1, x_2, x_3, x_4, x_2 + x_3 + x_4, x_1 + x_3 + x_4, x_1 + x_2 + x_4)$$

Note that they are binary additions. So $1 + 1 = 0$. It is basically XOR. Some definitions:

- **Hamming distance**: Hamming distance between $x, y \in \Sigma^n$ is the number of positions where x and y differ. It is denoted by $d(x, y) = \sum_{i=1}^{n} \mathbb{1}\{x_i \neq y_i\}$. So adding each bit that is different.
- **Relative distance**: counts for the length of the codeword. So $\frac{d(x,y)}{n}$.
- **Minimum distance**: The minimum distance between any two codewords. So $d_{min} = \min d(x, y)$, for which $x, y \in C, x \neq y$. It measures the robustness of the code. The larger the minimum distance, the more errors it can correct.

- **Hamming weight**: The number of non-zero elements in a vector. So $w(x) = \sum_{i=1}^{n} \mathbb{1}\{x_i \neq 0\}$. It is the number of 1s in the vector.

1.4 Hamming Code Properties

Theorem 1. *A code with a minimum distance of d can:*

- *Detect $d-1$ errors.*
- *Correct $\lfloor \frac{d-1}{2} \rfloor$ errors.*
- *Detect $d-1$ erasures.*
- *Correct $d-1$ erasures.*

1.5 Algorithm for Hamming Code

1.5.1 Error Detection

If given a codeword $\tilde{C}$ that we assume the error is within $d-1$ away from one point, we can calculate the syndrome S by $S = \tilde{C}H^T$. If $S = 0$, then there is no error. If $S \neq 0$, then there is an error.

1.5.2 Error Correction

For a codeword $\tilde{C}$, with an error smaller than or equal to $\lfloor \frac{d-1}{2} \rfloor$, we can recover the original codeword by finding the closest codeword to $\tilde{C}$. The distance will be within $\lfloor \frac{d-1}{2} \rfloor$.

Note that it is not possible to have two C that are within $\lfloor \frac{d-1}{2} \rfloor$ distance from $\tilde{C}$. Otherwise, the distance between the two C will be smaller than d. Here is the proof:

Proof. $\lfloor \frac{d-1}{2} \rfloor \times 2 \leq d-1 < d$. So circle of radius $\lfloor \frac{d-1}{2} \rfloor$ around $\tilde{C}$ does not contain any other codewords. So there is only one codeword that is within $\lfloor \frac{d-1}{2} \rfloor$ distance from $\tilde{C}$. □

To mathematically express this:
If $\tilde{C}$ has error $\leq \lfloor \frac{d-1}{2} \rfloor$, then return $C \in \mathcal{C}$ such that $d(\tilde{C}, C) \leq \lfloor \frac{d-1}{2} \rfloor$.

1.5.3 Erasure Correction

If we have a codeword $\tilde{C}$ with e erasures, where $e \leq d-1$, then we can recover the original codeword by finding the closest codeword to $\tilde{C}$. The distance will be within e.

1.6 More definitions

Definition 2. *The* ***message length/dimension*** *of $\mathcal{C}$ over an alphabet Σ is the length of the message. So $k = log_{|\sigma|}|\mathcal{C}|$.*

(The log base is the size of the alphabet set, and the log is the size of the code set)

Definition 3. *The* ***rate*** *of $\mathcal{C}$ is the ratio of the message length to the codeword length. So $R = \frac{k}{n}$.*

Note the rate is between 0 and 1.

- The closer it is to 1, the more efficient the code is.
- If $R = 1$, then the code is not redundant. So it is not an error-correcting code.
- The closer it is to 0, the more redundant the code is.
- If $R = 0$, then the code is not efficient. So it is not an error-correcting code.
- So we need to make trade-offs between efficiency and robustness.

We could find that R is a function: $R(d, n, k, \Sigma)$. So we can find the maximum rate and minimum rate for a given d, n, k, Σ. Basically, R has two bounds:

$$g(d, n, k, \Sigma) \leq R(d, n, k, \Sigma) \leq f(d, n, k, \Sigma)$$

1.7 Hamming Bound

Hamming bound is the upper bound of the rate of a code. It answers the question: How many Hamming balls can we put in Σ^n such that they are disjoint for a fixed d?

The question implies that the ball radius is $\lfloor \frac{d-1}{2} \rfloor$. With the given information we can find that

- We will have $|\mathcal{C}|$ disjoint balls of radius $\lfloor \frac{d-1}{2} \rfloor$.
- $\text{Vol}(\Sigma^n) = |\Sigma|^n$.
- $|\mathcal{C}| \leq \frac{|\Sigma|^n}{\text{Vol(Hamming ball)}}$.

Note that we are dealing with discrete points. So we need to use the floor function which actually just counts for the points within the ball.

1.7.1 Proof of Hamming Bound

Proof. The Hamming ball in Σ^n of radius e about $x \in \Sigma^n$ is the set of all points in Σ^n that are within e distance from x.

So $B_{\Sigma^n}(x, e) = \{y \in \Sigma^n : d(x, y) \leq e\}$ is the set of the discrete points that are within e distance from x, including x itself.

Now we can define the volume of the ball. The volume of the ball is the number of points in the ball. So $V_{\Sigma^n} = |B_{\Sigma^n}(x, e)|$ which is the size of the ballpoint set.

To make the calculation easier, we let $q = \Sigma^n$.

$$\text{Vol}_q(e, n) = 1 + \binom{n}{1}(q-1) + \binom{n}{2}(q-1)^2 + \ldots + \binom{n}{e}(q-1)^e = \sum_{i=0}^{e} \binom{n}{i}(q-1)^i$$

So since Hamming balls cannot overlap, we have:

$$\begin{aligned}
|\mathcal{C}| &\leq \frac{q^n}{\text{Vol}_q(e, n)} \\
|\mathcal{C}| \times \text{Vol}_q(e, n) &\leq q^n \\
\log_q(|\mathcal{C}|) + \log_q(\text{Vol}_q(e, n)) &\leq \log_q(q^n) \\
\log_q(|\mathcal{C}|) + \log_q(\text{Vol}_q(e, n)) &\leq n \\
\log_q(|\mathcal{C}|) &\leq n - \log_q(\text{Vol}_q(e, n)) \\
\textbf{Rate} = \frac{\log_q(|\mathcal{C}|)}{n} &\leq 1 - \frac{\log_q(\text{Vol}_q(e, n))}{n}
\end{aligned}$$

The rate here is the maximum rate. So we have the upper bound of the rate of a code. So we have the **Hamming bound**: □

1.7.2 Analysis of Hamming Bound for binary codes

Now that we have all the tools for binary Hamming codes:

- $q = 2$. So $|\Sigma| = 2$, which is the alphabet size.
- $n = 7$. So n is the length of the codeword.
- $k = 4$. So k is the length of the message.
- $d = 3$. So d is the minimum distance. Proof of this will be in the next section.

Using the above information, we find that the Hamming bound is:

$$R = 1 - \frac{\log_2(\mathrm{Vol}_2(\lfloor \frac{3-1}{2} \rfloor, 7))}{7} = 1 - \frac{\log_2(1+7)}{7} = \frac{4}{7}$$

But we also notice that the Hamming code has a rate of $\frac{k}{n} = \frac{4}{7}$. So the Hamming bound is tight. So the Hamming code is the best/optimal code for this case.

1.8 Linear Algebra View of Hamming Codes

If we view the encoding process as a matrix multiplication it will be like this:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_2 + x_3 + x_4 \\ x_1 + x_3 + x_4 \\ x_1 + x_2 + x_4 \end{bmatrix}$$

We can write it as $G \cdot X = \mathcal{C}$ where G is the generator matrix. So G is a $n \times k$ matrix. X is the message vector. $\mathcal{C}$ is the codeword vector. $\mathcal{C} = \{G \cdot X (\mod 2) : \forall x \in X, x \in \{0,1\}^4\}$

There are three properties with it:

1. $\mathcal{C}$ is closed under addition. So $\forall C_1, C_2 \in \mathcal{C}, C_1 + C_2 \in \mathcal{C}$.

Proof.

$$C_1 + C_2 = G \cdot X_1 + G \cdot X_2 = G \cdot (X_1 + X_2)$$

Since $X_1 + X_2 \in \{0,1\}^4$, so $C_1 + C_2 \in \mathcal{C}$. Proved. □

2. $\mathcal{C}$ is a linear subspace of $\{0,1\}^7$ of dimension 4.

   Proof. $\mathcal{C}$ is all the linear combinations of the columns of G. So $\mathcal{C}$ is a linear subspace of $\{0,1\}^7$ of dimension 4. Proved. □

3. The distance of $\mathcal{C}$ is the same as the minimum weight of the non-zero codewords in $\mathcal{C}$.

   Proof. Recall that $wt(x) = \sum_{i=1}^{n} \mathbb{1}\{x_i \neq 0\}$. So $wt(x)$ is the number of 1s in the vector.

   $\forall C_1, C_2 \in \mathcal{C}, C_1 \neq C_2$, then $wt(C_1 + C_2) = wt(C_1 \oplus C_2)$,which is just the number of different bits between C_1 and C_2. So $d(C_1, C_2) = wt(C_1 \oplus C_2)$.

   Thus if we go through all the possibilities space, we can claim that $d_{min} = \min_{C_1, C_2 \in \mathcal{C}, C_1 \neq C_2} d(C_1, C_2) = \min_{C \in \mathcal{C}, C \neq 0} wt(C)$. Proved. □

1.8.1 Parity Check Matrix

We can also define a parity check matrix H for the Hamming code. It is a $n \times (n-k)$ matrix. So $H \cdot C^T = 0$ where C is the codeword vector. So H is the matrix that can detect errors.

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ c_5 = x_2 + x_3 + x_4 \\ c_6 = x_1 + x_3 + x_4 \\ c_7 = x_1 + x_2 + x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Note that H is just letting the encoded codeword to XOR with its decomposition. It should be 0 if there is no error. So if there is an error, then $H \cdot C^T \neq 0$.

1.8.2 Proof of Minimum Distance

With the help of the parity check matrix, we can prove that the minimum distance of the Hamming code is 3.

Proof. Proof by contradiction: Suppose that $\exists C \in \mathcal{C}$ such that $wt(C) = 1$ or $wt(C) = 2$.

We can list out all the possibilities of C and find out that: $H \cdot C^T \neq 0$. So C is not a codeword. Contradiction. So $\forall C \in \mathcal{C}, wt(C) \geq 3$.

Now just need to find a codeword with weight 3. We can find that $C = (1, 1, 1, 0, 0, 0, 0)$ is a codeword. With the property that we just proved:

The distance of $\mathcal{C}$ is the same as the minimum weight of the non-zero codewords in $\mathcal{C}$.

So $d_{min} = 3$. Proved. □

1.9 Decoding Hamming Codes

Assume that we have a codeword $\tilde{C}$ that is transmitted. We want to find the closest codeword to $\tilde{C}$.

- If $\tilde{C}$ has no error, then we can just return $\tilde{C}$.
- If $\tilde{C}$ has one error, $\tilde{C} = C \oplus Z$ where Z is the noise such that $wt(Z) = 1$.

$$\begin{aligned} H \cdot \tilde{C}^T &= H \cdot (C \oplus Z)^T \\ &= H \cdot C^T \oplus H \cdot Z^T && \text{Addition is commutative} \\ &= H \cdot Z^T && \text{Since } H \cdot C^T = 0 \end{aligned}$$

  From there, we can know which bit is wrong now, and moreover, we can correct it.

$$C = \tilde{C} \oplus Z$$

  XOR can be reversed by XOR again. So we can correct the error.

2 General Linear Block Codes

2.1 Finite Fields

Intuitively, a (finite) field is a (finite) set of elements that can be added, subtracted, multiplied, and divided.

Formally, we can define as follows:

Definition 4. *A field $\mathbb{F}$ is a set of elements, along with operations of addition and multiplication, such that $\forall x, y, z \in \mathbb{F}$*

- *(Associativity) $x + (y + z) = (x + y) + z$ and $x \cdot (y \cdot z) = (x \cdot y) \cdot z$.*
- *(Commutativity) $x + y = y + x$ and $x \cdot y = y \cdot x$.*
- *(Distributivity) $x \cdot (y + z) = x \cdot y + x \cdot z$.*
- *(Identity):*
  - *$\exists$'$0' \in \mathbb{F}$ such that $x + 0 = x$.*
  - *$\exists$'$1' \in \mathbb{F}$ such that $x \cdot 1 = x$.*
- *(Inverse):*
  - *$\forall x \in \mathbb{F}, \exists -x \in \mathbb{F}$ such that $x + (-x) = 0$.*
  - *$\forall x \in \mathbb{F}, x \neq 0, \exists x^{-1} \in \mathbb{F}$ such that $x \cdot x^{-1} = 1$.*

2.1.1 Existance of finite fields

We can examine two examples first: $\mathbb{F}_1 = \{0, 1, 2, 3, 4\}$ with '+' = '+ mod 5','×' = '× mod 5' and $\mathbb{F}_2 = \{0, 1, 2, 3\}$ with '+'= '+ mod 4','×' = '× mod 4'.

It is easy to verify associativity, commutativity, distributivity and identity for both, but let's take a closer look into the inverse.

- $\mathbb{F}_1$:
  - 0 has no inverse for multiplication.
  - $1, 2, 3, 4$ all have inverses for multiplication.
  - $0, 1, 2, 3, 4$ all have inverses for addition.
- $\mathbb{F}_2$:
  - 0 has no inverse for multiplication.
  - $1, 3$ all have inverses for multiplication. But what is the inverse of 2? There is no such thing. **So $\mathbb{F}_2$ is not a field.**
  - $0, 1, 2, 3$ all have inverses for addition.

Theorem 2. *When p is a prime number, then $\mathbb{F}_p = \{0, 1, 2, \ldots, p-1\}$ with '+'= '+ mod p','×' = '× mod p' is a field.*

2.2 Linear algebra's view of finite fields

Let $\mathbb{F}$:

- $\mathbb{F}^n = \{(x_1, x_2, \ldots, x_n) : x_i \in \mathbb{F}\}$.
- A **subspace** $V \subseteq \mathbb{F}^n$ is a set of vectors that is closed under addition and scalar multiplication. So $\forall u, v \in V, \lambda \in \mathbb{F}, u + v \in V, \lambda u \in V$.
- $v_1, v_2, \ldots, v_k \in \mathbb{F}^n$ are **linearly independent** if $\lambda_1 v_1 + \lambda_2 v_2 + \ldots + \lambda_k v_k = 0$ implies that $\lambda_1 = \lambda_2 = \ldots = \lambda_k = 0$.
- For $v_1, v_2, \ldots, v_k \in \mathbb{F}^n$, the **span** of $v_1, v_2, \ldots, v_k$ is the set of all linear combinations of $v_1, v_2, \ldots, v_k$. So $\text{span}(v_1, v_2, \ldots, v_k) = \{\lambda_1 v_1 + \lambda_2 v_2 + \ldots + \lambda_k v_k : \lambda_1, \lambda_2, \ldots, \lambda_k \in \mathbb{F}\} = \{\sum_i \lambda_i v_i : \lambda \in \mathbb{F}\}$.
- A **basis** for a subspace $V \subseteq \mathbb{F}^n$ is a set of linearly independent vectors that span V.
- The **dimension** of a subspace $V \subseteq \mathbb{F}^n$ is the number of vectors in a basis for V.

2.3 Linear Block Codes over Finite Fields

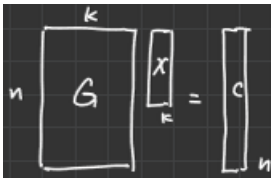
Definition 5. *A linear block code $\mathcal{C}$ of length n and dimension k over a finite field $\mathbb{F}$ is a $k-$dimension linear subspace of $\mathbb{F}^n$. So $\mathcal{C} \subseteq \mathbb{F}^n$. The elements $c \in \mathcal{C}$ are called codewords.*

Note that if we connect to previous definitions, it is the same as block length n and message length k over alphabet $\Sigma = \mathbb{F}$.

$$\mathcal{C} \text{ is a "linear code"} \implies \mathcal{C} \text{ is a linear subspace of } \mathbb{F}^n$$

2.3.1 Generator Matrix

Definition 6. *Let $\mathcal{C} \subseteq \mathbb{F}^n$ be a linear block code of length n and dimension k. A generator matrix G for $\mathcal{C}$ is a $n \times k$ matrix such that $\mathcal{C} = \{G \cdot X : X \in \mathbb{F}^k\}$.*



$$[G] = \begin{bmatrix} I_k \\ A \end{bmatrix}$$

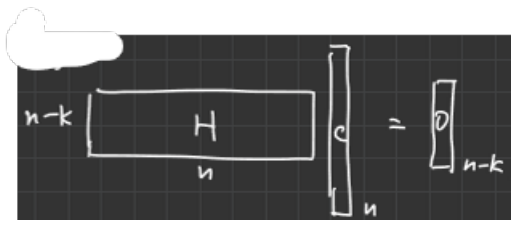
Observations:

- Any linear code has a generator matrix. (Choose any basis for the code and put the basis vectors as the columns of the generator matrix.)
- The generator matrix is not unique. (Different bases will give different generator matrices.)

A generator like the above is called a **systematic generator matrix** where I_k is the identity matrix of size k and A is a $(n-k) \times k$ matrix. It is called systematic because the message is part of the codeword.

2.3.2 Parity Check Matrix

Definition 7. *Let $\mathcal{C} \subseteq \mathbb{F}^n$ be a linear block code of length n and dimension k. A parity check matrix $H \in \mathbb{F}^{(n-k)\times k}$ for $\mathcal{C}$ is a $(n-k) \times n$ matrix such that $\mathcal{C} = \{C \in \mathbb{F}^n : H \cdot C = 0\}$. H is the kernel matrix of G.*



$[H] = \begin{bmatrix} B_{(n-k)\times k} & I_k \end{bmatrix}$
Observations:

- Any linear code has a parity check matrix.
- The parity check matrix is not unique.

A parity check matrix like the above is called a **systematic parity check matrix** where $I_{(n-k)}$ is the identity matrix of size $(n-k)$ and B is a $(n-k)\times k$ matrix. It is called systematic because the message is part of the codeword.

2.3.3 Relationship between G and H

Question:

Let $\mathcal{C}$ be a linear code of dimension k and length n over $\mathbb{F} = \{0, 1\}$. Let G be a generator matrix for $\mathcal{C}$ and H be a parity check matrix for $\mathcal{C}$.

What is the relationship between G and H, or more specifically what is the relationship between A and B?

Let the original message be X. Note that $A \cdot X$ is the parity bits.

Then we can write the codeword as $C = G \cdot X = \begin{bmatrix} I_k \\ A \end{bmatrix} \cdot X = \begin{bmatrix} X \\ (A \cdot X) \end{bmatrix}$. Also, we know that $H \cdot C = 0$. So

$$\begin{bmatrix} B_{(n-k)\times k} & I_k \end{bmatrix} \cdot \begin{bmatrix} X_{k\times k} \\ (A \cdot X)_{(n-k)\times k} \end{bmatrix} = 0 \implies B \cdot X + I \cdot (A \cdot X) = 0$$

So $B + A = 0$ note that the addition is the addition defined in the field $\mathbb{F} = \{0, 1\}$. So $B + A = 0 \implies B = A$ in a binary sense.